Rovem — Data Model

Oct 11, 2026 · @k._muz

Rovem stores optional player accounts and saved progress in seven PostgreSQL tables: three hold fixed game content, one holds the player, and three hold that player's progress. Companion to the Software Requirements Specification, Draft 0.5 (FR-20 to FR-25, FR-35 to FR-37, FR-41 to FR-44).

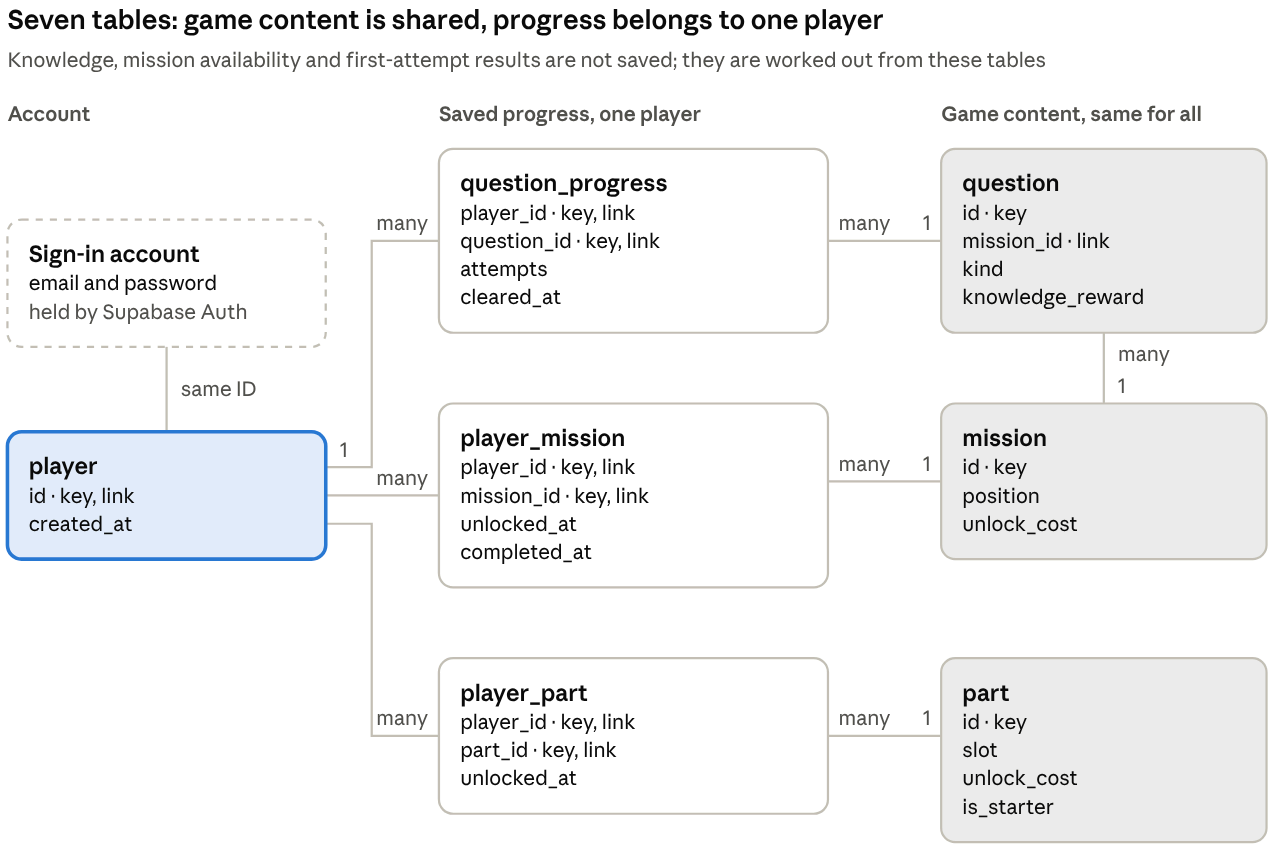
Database choice

Rovem uses PostgreSQL, hosted on Supabase's free plan.

- **Standard relational database.** Tables and links map directly onto the ER diagram below, and the schema can move to any PostgreSQL host without redesign.
- **Sign-in included.** Email-and-password accounts with password reset come built in (FR-41), so no sign-in system is written by hand.
- **Per-player access rules.** Row-level security lets the database itself ensure each player reads and changes only their own rows (FR-42).
- **Free plan fits the MVP.** It includes a 500 MB database and 50,000 monthly active users (Supabase pricing).
- **Known limit.** Free projects pause after one week without activity and must be restored from the dashboard. The project is restored and checked before every demonstration.

Alternatives considered: Firebase stores documents rather than tables, so it has no ER diagram or SQL schema to derive. Neon offers PostgreSQL alone, so sign-in would have to be built separately.

ER diagram



Rovem data model · 7 tables, 1 sign-in account

How to read it. Each box is one table, and each line inside it is one piece of information the table holds. "key" marks the information that tells rows apart; "link" marks information pointing to a row in another table. A line between two boxes joins linked tables, with "1" beside the table that has one matching row and "many" beside the table that has many. For example, one player has many question_progress rows, and many questions belong to one mission. In the progress tables, the two keys together tell rows apart, so a student has at most one row per question, mission or part. The shaded boxes on the right are game content, the same for every student. The highlighted box is the student, and the dashed box is the sign-in account that Supabase holds, sharing the student's ID.

Tables in plain words

Content tables hold only what the database must check: IDs, order, costs and rewards. The game's content files remain the source for these numbers. A seeding step copies them into the tables, and an automated test checks that the two agree (NFR-5). Text,

physics values and explanations stay in the files.

Table	One row is	Holds
mission	One mission	Its order (1 to 3) and its unlock cost: 0 for Mission 1, 20 for Missions 2 and 3
part	One rover part	Its slot (chassis, wheels or motor), unlock cost (15, or 0 for a starting part) and whether it is a starting part
question	One question, original and variant together	Its mission, kind (required, power-up or unlock) and Knowledge reward (10 or 5)
player	One signed-in student	The account ID and when it was created; the email and password are held by Supabase's sign-in service
question_progress	One question a student has answered at least once	How many times it has been answered, and when it was cleared (empty while it is to review)
player_mission	One mission a student has unlocked; Mission 1 is added at sign-up	When it was unlocked and when it was completed (empty until then)
player_part	One part a student has unlocked; the three starting parts are added at sign-up	When it was unlocked

Worked out when needed, not saved

Six values are worked out from the tables whenever they are needed, so they can never disagree with the data they come from.

Value	How it is calculated	Requirement
Knowledge balance	Rewards of all cleared questions, minus the unlock costs of all unlocked missions and parts	FR-35, FR-36
Answered correctly first time	The question is cleared and has exactly one attempt	FR-23
Original or variant shown next	While the question is not cleared: the original after an even number of attempts, the variant after an odd number	FR-21
Mission available	The mission is unlocked and the previous mission is completed	FR-36
Unlock question available	That mission's three required questions have each been answered at least once	FR-34
Unanswered question	The question has no row in question_progress	FR-20

The balance formula:

$$\text{Knowledge} = \sum_{\text{cleared}} \text{reward} \; - \sum_{\text{unlocked}} \text{cost}$$

Rules the database enforces

- **Own data only.** A student can read only rows carrying their own account ID. They can add and update their own answers and mission completions, but not their own unlocks (FR-42).
- **Unlocks go through one database function.** It checks that the balance covers the cost and, for a mission, that the previous mission is completed, then records the unlock in a single step. If either check fails, nothing changes, so Knowledge never goes below zero (FR-44).
- **Progress only moves forward.** A cleared question stays cleared, and a completed mission stays completed (FR-20, FR-24).
- **New accounts start with the free items.** Mission 1 and the three starting parts are recorded as unlocked at no cost (FR-37, FR-41).

- **Deleting an account deletes its progress.** Removing the player removes all of their progress rows. Deleting a sign-in account needs Supabase's secret service key, which must never reach the browser, so it runs in one small server-side function (FR-41).
- **Accepted limit.** The game reports each answer, so a determined student could alter their own progress. This affects only their own game, because Rovem has no leaderboard.

Guests and offline play

The browser keeps a copy of the same progress in local storage, in the same shape as the progress tables.

- **Guests** play with the local copy only and have no rows in the database.
- **First sign-in.** If the account is empty, the guest's progress is copied into it. If the account already has progress, the student chooses which of the two to keep, so Knowledge is never counted twice (FR-43).
- **Same account on two devices.** For each question, the higher attempt count and the earlier clearing time are kept (FR-43).
- **Signed in without a connection.** Answers and mission completions are saved locally and sent to the account when the connection returns. Unlocks need a connection, so every purchase passes the balance check (FR-44).
- **Account emails.** Supabase's built-in email sender is meant for testing and has a very low hourly limit (sources quote 2 to 4 emails an hour), so sign-up and password-reset emails for a class need a custom email (SMTP) provider, which the free plan allows.

What is not stored

Run results, run predictions and the current rover build are shown during play but not saved, because no requirement asks for them (FR-14 to FR-16, FR-32). A run table can be added later for playtest analysis or the future AI helper without changing the tables above.

Sources

- Supabase pricing: free plan database size, monthly active users, inactivity pausing, custom SMTP and Edge Functions.

- Supabase reference: auth.admin.deleteUser: deleting a user needs the service_role key and must run on a server.
- Makerkit: Supabase authentication for production: built-in email limit and custom SMTP.